

Movie Ticket Booking System

Master Test Case Document

End-to-end test coverage: from Level 0 smoke tests

Spring Boot 3.3 - Java 21 - PostgreSQL + Flyway

Generated: 08 July 2026

1. Introduction & Scope

This document is the master test-case catalogue for the Movie Ticket Booking System backend. It is organised as a progression of maturity levels, starting at Level 0 (environment smoke tests) and building up to advanced, concurrency-sensitive, end-to-end business scenarios.

The single most important invariant of the system is: NO SEAT IS EVER SOLD TWICE. A large share of the advanced test cases exist specifically to prove that invariant holds under concurrent load, database contention, and hold expiry.

The 'why' behind the levels

- Level 0 - Smoke: prove the app boots, schema migrates, and docs are reachable.
- Level 1 - Authentication: registration and login correctness.
- Level 2 - Authorisation / RBAC: who can call what (ADMIN vs CUSTOMER vs anonymous).
- Level 3 - Input Validation: bean-validation on every request DTO.
- Level 4 - Admin Catalog: cities, theaters, screens, seats, movies.
- Level 5 - Pricing / Discount / Refund configuration.
- Level 6 - Show management and public browsing/search.
- Level 7 - Booking lifecycle happy paths (hold -> confirm -> cancel).
- Level 8 - Booking negative & edge cases.
- Level 9 - Hold expiry (scheduler + confirm-time).
- Level 10 - Concurrency (the crown jewel): one seat, many buyers, one winner.
- Level 11 - End-to-end journeys and non-functional checks.

Priority legend

- P0 - Critical. A failure blocks release (correctness / money / double-sell).
- P1 - High. Core functionality; must pass before sign-off.
- P2 - Medium. Validation, edge cases, and nice-to-have coverage.

2. System Under Test

A concurrency-safe REST backend for booking cinema seats. Layering is strictly controller -> service -> repository. Controllers are thin; all business logic and @Transactional boundaries live in the service layer.

2.1 Key components

Area	Detail
Auth	JWT bearer tokens; roles ADMIN and CUSTOMER; stateless.
Booking core	hold -> confirm -> cancel, all @Transactional.
Seat locking	Pessimistic SELECT ... FOR UPDATE in deterministic id order.
Optimistic lock	@Version on ShowSeat and Booking.
DB backstop	Partial unique index uq_booking_seat_active on booking_seat(show_seat_id) WHERE active = true.
Hold expiry	TTL (default 120s) swept by scheduler + checked at confirm.
Pricing	base x seatClass multiplier x weekend multiplier.
Notifications	@TransactionalEventListener(AFTER_COMMIT) + @Async.
Errors	ApiException + GlobalExceptionHandler + ErrorCode enum.

2.2 Booking state machine

```

PENDING --confirm--> CONFIRMED --cancel(refund=0)--> CANCELLED
                                --cancel(refund>0)--> REFUNDED
PENDING --hold TTL exceeded--> EXPIRED

ShowSeat: AVAILABLE <--> HELD -> BOOKED
           (cancel / expiry releases HELD or BOOKED back to AVAILABLE)

```

2.3 Error codes reference

ErrorCode	HTTP	Raised when
VALIDATION_ERROR	400	Bean validation / seat-show mismatch
INVALID_DISCOUNT	400	Bad / inactive / expired / below-min discount
UNAUTHORIZED	401	Missing or invalid authentication
INVALID_CREDENTIALS	401	Wrong email/password on login
FORBIDDEN	403	Wrong role or not the booking owner
NOT_FOUND	404	Entity id does not exist
CONFLICT	409	Wrong booking state / duplicate city or code
SEAT_UNAVAILABLE	409	Seat not AVAILABLE / concurrency loss
HOLD_EXPIRED	409	Confirm attempted after hold TTL
REFUND_NOT_ALLOWED	409	Cancel of a non-CONFIRMED booking
EMAIL_ALREADY_USED	409	Duplicate registration email
INTERNAL_ERROR	500	Unhandled exception

3. Test Environment & Manual Testing Guide

Every test case below has a 'How to test manually' section. This chapter describes the one-time setup so those instructions work as-is.

3.1 Start the application

- Ensure PostgreSQL is running. Quickest: 'docker compose up -d' from repo root.
- Build without tests: `mvn -B -DskipTests` package
- Run: `mvn spring-boot:run` (defaults to `http://localhost:8080`)
- Flyway applies V1 schema + V2 seed data automatically on startup.
- With `demo.seed=true` (default), an admin, a customer and a demo catalog are seeded.

3.2 Seeded credentials (demo.seed=true, empty DB)

Role	Email	Password
ADMIN	admin@movies.test	admin123
CUSTOMER	customer@movies.test	customer123

3.3 Tooling options for manual testing

- Swagger UI (recommended for exploration): `http://localhost:8080/swagger-ui.html` - lets you Authorize with a bearer token then click Execute on any endpoint.
- Postman / Insomnia: import the endpoints, set a collection variable `{{token}}`, add header 'Authorization: Bearer `{{token}}`'.
- curl (used in this document): copy/paste the snippets. On Windows PowerShell prefer `curl.exe` to avoid the `Invoke-WebRequest` alias.
- Database inspection (psql / DBeaver): verify `show_seat.status`, `booking.status`, `booking_seat.active`, `payment.status` directly.
- Automated: `mvn test` runs the JUnit suite including the concurrency race test.

3.4 Get a token (used by most manual steps)

Log in and capture the JWT. All authenticated calls reuse it as `$TOKEN` / `$ADMIN_TOKEN`.

```
# Customer token
curl -s -X POST http://localhost:8080/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"customer@movies.test","password":"customer123"}'
# -> copy the "token" value into $TOKEN

# Admin token
curl -s -X POST http://localhost:8080/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@movies.test","password":"admin123"}'
# -> copy the "token" value into $ADMIN_TOKEN
```

Tip: to discover a live showId and its show-seat ids for booking cases, call `GET /shows` and `GET /shows/{showId}/seats` (both public).

4. Level 0 - Smoke & Environment Tests

Goal: prove the environment is healthy before any functional testing.

P0 TC-0.1 Application boots successfully

Type	Smoke / Non-functional
Preconditions	PostgreSQL running; ports free.

Steps:

1. Start the app with `mvn spring-boot:run`.
2. Watch the startup log.

Expected result:

- Log shows 'Started MovieBookingApplication'.
- No stack traces; Tomcat listening on 8080.

How to test manually:

```
curl -s -o NUL -w "%{http_code}\n" http://localhost:8080/swagger-ui.html
# expect 200 (or 302 redirect to the UI)
```

P0 TC-0.2 Flyway migrations apply and schema validates

Type	Smoke / DB
Preconditions	Fresh database.

Steps:

1. Start the app against an empty schema.
2. Inspect the `flyway_schema_history` table.

Expected result:

- `V1__init_schema` and `V2__seed_reference_data` are marked success.
- Hibernate `ddl-auto=validate` passes (app does not fail to start).
- 14 tables exist (`app_user`, `city`, `theater`, `screen`, `seat`, `movie`, `show_event`, `pricing_tier`, `discount_code`, `refund_policy`, `show_seat`, `booking`, `booking_seat`, `payment`, `audit_event`).

How to test manually:

```
psql moviebooking -c "select version, description, success from flyway_schema_history order by
installed_rank;"
```

P2 TC-0.3 Swagger / OpenAPI docs are reachable

Type	Smoke
Preconditions	App running.

Steps:

1. Open the OpenAPI JSON and the Swagger UI.

Expected result:

- GET `/v3/api-docs` returns 200 with a valid OpenAPI document.
- Swagger UI renders all controllers and an Authorize button.

How to test manually:

```
curl -s -o NUL -w "%{http_code}\n" http://localhost:8080/v3/api-docs
# expect 200
```

P1	TC-0.4 Reference seed data present
Type	Smoke / DB
Preconditions	App started once.

Steps:

1. Query pricing_tier and refund_policy tables.

Expected result:

- pricing_tier has REGULAR=1.0, PREMIUM=1.5, RECLINER=2.0.
- refund_policy 'Standard Policy' with full=24h, partial=4h @ 50%, active=true.

How to test manually:

```
psql moviebooking -c "select seat_class, multiplier from pricing_tier; select * from refund_policy;"
```

5. Level 1 - Authentication

Endpoints: POST /auth/register, POST /auth/login (both public).

P0 TC-1.1 Register a new customer (happy path)

Type	Functional
Preconditions	Email not already used.

Steps:

1. POST /auth/register with a valid email, password (>=6 chars) and fullName.

Test data:

{ email: alice@test.com, password: secret1, fullName: Alice }

Expected result:

- 201 Created; body contains a JWT token and role=CUSTOMER.
- New row in app_user with hashed password (never plaintext).
- Self-registration always yields CUSTOMER (never ADMIN).

How to test manually:

```
curl -i -X POST http://localhost:8080/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"alice@test.com","password":"secret1","fullName":"Alice"}'
```

P0 TC-1.2 Login with valid credentials

Type	Functional
Preconditions	User exists (e.g. seeded customer).

Steps:

1. POST /auth/login with correct email/password.

Expected result:

- 200 OK; body contains a valid JWT usable on protected endpoints.

How to test manually:

```
curl -i -X POST http://localhost:8080/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"customer@movies.test","password":"customer123"}'
```

P1 TC-1.3 Register with invalid email format

Type	Validation
Preconditions	None.

Steps:

1. POST /auth/register with email='not-an-email'.

Expected result:

- 400 Bad Request; code=VALIDATION_ERROR.
- fieldErrors contains an entry for 'email'.

How to test manually:

```
curl -i -X POST http://localhost:8080/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"not-an-email","password":"secret1","fullName":"Bob"}'
```

P1	TC-1.4 Register with too-short password
Type	Validation
Preconditions	None.

Steps:

1. POST /auth/register with a 3-char password.

Expected result:

- 400; VALIDATION_ERROR; fieldErrors mentions 'password' (@Size min=6).

How to test manually:

```
curl -i -X POST http://localhost:8080/auth/register \
-H "Content-Type: application/json" \
-d '{"email":"bob@test.com","password":"abc","fullName":"Bob"}'
```

P2	TC-1.5 Register with missing required fields
Type	Validation
Preconditions	None.

Steps:

1. POST /auth/register with an empty body {}.

Expected result:

- 400; VALIDATION_ERROR; fieldErrors for email, password, fullName (@NotBlank).

How to test manually:

```
curl -i -X POST http://localhost:8080/auth/register \
-H "Content-Type: application/json" -d '{}'
```

P1	TC-1.6 Register with an already-used email
Type	Negative / Conflict
Preconditions	Email already registered (e.g. customer@movies.test).

Steps:

1. POST /auth/register reusing an existing email.

Expected result:

- 409 Conflict; code=EMAIL_ALREADY_USED.

How to test manually:

```
curl -i -X POST http://localhost:8080/auth/register \
-H "Content-Type: application/json" \
-d '{"email":"customer@movies.test","password":"secret1","fullName":"Dup"}'
```

P1	TC-1.7 Login with wrong password
Type	Negative
Preconditions	User exists.

Steps:

1. POST /auth/login with correct email, wrong password.

Expected result:

- 401 Unauthorized; code=INVALID_CREDENTIALS; no token issued.

How to test manually:

```
curl -i -X POST http://localhost:8080/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"customer@movies.test","password":"wrongpass"}'
```

P2 TC-1.8 Login with unknown email	
Type	Negative
Preconditions	Email not registered.

Steps:

1. POST /auth/login with a non-existent email.

Expected result:

- 401; INVALID_CREDENTIALS (message must not reveal whether email exists).

How to test manually:

```
curl -i -X POST http://localhost:8080/auth/login \  
-H "Content-Type: application/json" \  
-d '{"email":"ghost@test.com","password":"whatever"}'
```

6. Level 2 - Authorisation & RBAC

Verifies who may call what. Public: /auth/**, GET /shows/**, swagger. CUSTOMER: booking endpoints. ADMIN: /admin/**.

P0 TC-2.1 Anonymous cannot access admin endpoint

Type	Security
Preconditions	No Authorization header.

Steps:

1. Call an /admin endpoint without a token.

Expected result:

- 401 Unauthorized; code=UNAUTHORIZED.

How to test manually:

```
curl -i http://localhost:8080/admin/cities
```

P0 TC-2.2 CUSTOMER forbidden on admin endpoints

Type	Security
Preconditions	Valid CUSTOMER token (\$TOKEN).

Steps:

1. Call an /admin endpoint with a customer token.

Expected result:

- 403 Forbidden; code=FORBIDDEN.

How to test manually:

```
curl -i http://localhost:8080/admin/cities \
-H "Authorization: Bearer $TOKEN"
```

P1 TC-2.3 ADMIN forbidden on customer booking endpoints

Type	Security
Preconditions	Valid ADMIN token (\$ADMIN_TOKEN).

Steps:

1. Call GET /bookings/me with an admin token (endpoint requires ROLE_CUSTOMER).

Expected result:

- 403 Forbidden; code=FORBIDDEN (role separation enforced).

How to test manually:

```
curl -i http://localhost:8080/bookings/me \
-H "Authorization: Bearer $ADMIN_TOKEN"
```

P0 TC-2.4 Anonymous cannot create a hold

Type	Security
Preconditions	None.

Steps:

1. POST /shows/{id}/holds without a token.

Expected result:

- 401 Unauthorized.

How to test manually:

```
curl -i -X POST http://localhost:8080/shows/1/holds \
-H "Content-Type: application/json" -d '{"showSeatIds":[1]}'
```

P1	TC-2.5 Malformed / tampered JWT is rejected
Type	Security
Preconditions	None.

Steps:

1. Send a protected request with a garbage bearer token.

Expected result:

- 401 Unauthorized; request never reaches the controller.

How to test manually:

```
curl -i http://localhost:8080/bookings/me \
-H "Authorization: Bearer not.a.valid.jwt"
```

P2	TC-2.6 Expired JWT is rejected
Type	Security
Preconditions	A token past app.jwt.expiration-minutes (temporarily lower the config, or wait).

Steps:

1. Call a protected endpoint with an expired token.

Expected result:

- 401 Unauthorized.

How to test manually:

Set app.jwt.expiration-minutes to a tiny value in a test profile, log in, wait, then:

```
curl -i http://localhost:8080/bookings/me -H "Authorization: Bearer $EXPIRED"
```

P1	TC-2.7 Public browse allowed without token
Type	Security
Preconditions	None.

Steps:

1. GET /shows without any Authorization header.

Expected result:

- 200 OK; list returned (read-only browsing is public).

How to test manually:

```
curl -i http://localhost:8080/shows
```

7. Level 3 - Input Validation

Bean-validation must reject malformed request bodies with 400 VALIDATION_ERROR and a fieldErrors array, before any business logic runs.

DTO	Field	Rule to violate
HoldRequest	showSeatIds	@NotEmpty - send empty list
ShowDto.Request	basePrice	@Positive - send 0 or negative
ShowDto.Request	movieId/screenId/startTime	@NotNull - omit them
CityDto.Request	name/state	@NotBlank - send blanks
TheaterDto.Request	cityId	@NotNull - omit
SeatDto.RowSpec	seatCount	@Min(1) - send 0
MovieDto.Request	durationMinutes	@NotNull @Min(1)
DiscountCodeDto.Request	value	@Positive
PricingTierDto.Request	multiplier	@DecimalMin(0.0)
RefundPolicyDto.Request	partialRefundPercent	@DecimalMin/Max 0..100

P1 TC-3.1 Hold with empty seat list rejected

Type	Validation
Preconditions	Valid CUSTOMER token; valid showId.

Steps:

1. POST /shows/{id}/holds with {"showSeatIds":[]}

Expected result:

- 400; VALIDATION_ERROR; fieldErrors for showSeatIds (@NotEmpty).

How to test manually:

```
curl -i -X POST http://localhost:8080/shows/1/holds \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"showSeatIds":[]}'
```

P1 TC-3.2 Create show with non-positive base price

Type	Validation
Preconditions	Valid ADMIN token.

Steps:

1. POST /admin/shows with basePrice = 0.

Expected result:

- 400; VALIDATION_ERROR; fieldErrors for basePrice (@Positive).

How to test manually:

```
curl -i -X POST http://localhost:8080/admin/shows \
  -H "Authorization: Bearer $ADMIN_TOKEN" -H "Content-Type: application/json" \
  -d '{"movieId":1,"screenId":1,"startTime":"2027-01-01T18:00:00","basePrice":0}'
```

P2 TC-3.3 Content-Type / malformed JSON handling

Type	Validation
Preconditions	Valid token.

Steps:

1. POST a body that is not valid JSON.

Expected result:

-
- 400 Bad Request; consistent ApiError shape (no stack trace leaked).

How to test manually:

```
curl -i -X POST http://localhost:8080/auth/register \  
-H "Content-Type: application/json" -d '{ this is not json '}
```

8. Level 4 - Admin Catalog Management

Endpoints under /admin (ADMIN only). Build the physical catalog hierarchy: City -> Theater -> Screen -> Seats, plus Movies.

P1 TC-4.1 Create a city

Type	Functional
Preconditions	ADMIN token.

Steps:

1. POST /admin/cities with a unique name and state.

Expected result:

- 201 Created; response contains generated id.

How to test manually:

```
curl -i -X POST http://localhost:8080/admin/cities \
  -H "Authorization: Bearer $ADMIN_TOKEN" -H "Content-Type: application/json" \
  -d '{"name":"Bengaluru","state":"Karnataka"}'
```

P2 TC-4.2 Duplicate city name rejected

Type	Negative / Conflict
Preconditions	City 'Bengaluru' already exists.

Steps:

1. POST /admin/cities with the same name again.

Expected result:

- 409 Conflict; code=CONFLICT (city.name is UNIQUE).

How to test manually:

```
curl -i -X POST http://localhost:8080/admin/cities \
  -H "Authorization: Bearer $ADMIN_TOKEN" -H "Content-Type: application/json" \
  -d '{"name":"Bengaluru","state":"Karnataka"}'
```

P1 TC-4.3 Create theater, screen and seat layout

Type	Functional
Preconditions	ADMIN token; a cityId exists.

Steps:

1. POST /admin/theaters (with cityId).
2. POST /admin/screens (with theaterId).
3. POST /admin/screens/{screenId}/seats with a row layout.

Test data:

seats: rows=[{rowLabel:A, seatCount:10, seatClass:REGULAR}, {rowLabel:B, seatCount:8, seatClass:PREMIUM}]

Expected result:

- Each POST returns 201 with an id.
- Seat layout returns the created seats; UNIQUE(screen_id,row_label,seat_number) enforced.

How to test manually:

```
curl -i -X POST http://localhost:8080/admin/theaters -H "Authorization: Bearer $ADMIN_TOKEN" -H
  "Content-Type: application/json" -d '{"name":"PVR","address":"MG Road","cityId":1}'
```

```
curl -i -X POST http://localhost:8080/admin/screens -H "Authorization: Bearer $ADMIN_TOKEN" -H
  "Content-Type: application/json" -d '{"name":"Screen 1","theaterId":1}'
```

```
curl -i -X POST http://localhost:8080/admin/screens/1/seats -H "Authorization: Bearer $ADMIN_TOKEN" -H
  "Content-Type: application/json" -d
  '{"rows":[{"rowLabel":"A","seatCount":10,"seatClass":"REGULAR"}]}'
```

P1	TC-4.4 Create a movie
Type	Functional
Preconditions	ADMIN token.

Steps:

1. POST /admin/movies with title, language, genre, durationMinutes.

Expected result:

- 201 Created; movie id returned.

How to test manually:

```
curl -i -X POST http://localhost:8080/admin/movies \  
-H "Authorization: Bearer $ADMIN_TOKEN" -H "Content-Type: application/json" \  
-d \  
'{"title":"Inception","language":"English","genre":"SciFi","durationMinutes":148,"certification":"UA"  
"}'
```

P2	TC-4.5 Create entity with non-existent parent id
Type	Negative
Preconditions	ADMIN token.

Steps:

1. POST /admin/theaters with cityId=999999 (missing).

Expected result:

- 404 Not Found; code=NOT_FOUND.

How to test manually:

```
curl -i -X POST http://localhost:8080/admin/theaters \  
-H "Authorization: Bearer $ADMIN_TOKEN" -H "Content-Type: application/json" \  
-d '{"name":"Ghost","address":"Nowhere","cityId":999999}'
```

9. Level 5 - Pricing, Discount & Refund Configuration

P1 TC-5.1 Create and update a pricing tier

Type	Functional
Preconditions	ADMIN token.

Steps:

1. POST /admin/pricing-tiers with seatClass + multiplier.
2. PUT /admin/pricing-tiers/{id} to change the multiplier.

Expected result:

- 201 on create, 200 on update; GET reflects the new multiplier.

How to test manually:

```
curl -i -X POST http://localhost:8080/admin/pricing-tiers -H "Authorization: Bearer $ADMIN_TOKEN" -H "Content-Type: application/json" -d '{"name":"Premium","seatClass":"PREMIUM","multiplier":1.5}'
```

```
curl -i http://localhost:8080/admin/pricing-tiers -H "Authorization: Bearer $ADMIN_TOKEN"
```

P0 TC-5.2 Pricing formula correctness (unit)

Type	Unit
Preconditions	None (PricingServiceTest).

Steps:

1. Compute price for REGULAR weekday, PREMIUM weekday, and PREMIUM weekend.

Test data:

base=100; PREMIUM=1.5x; weekend=1.25x.

Expected result:

- REGULAR weekday = 100.00.
- PREMIUM weekday = 150.00.
- PREMIUM weekend = 187.50 (rounded HALF_UP, 2 dp).
- Missing tier defaults multiplier to 1.0.

How to test manually:

```
mvn -Dtest=PricingServiceTest test
```

P1 TC-5.3 Create a discount code

Type	Functional
Preconditions	ADMIN token.

Steps:

1. POST /admin/discount-codes with code, type, value, caps and validity.

Test data:

SAVE20 = PERCENT 20, maxDiscount=100, minAmount=300, active=true.

Expected result:

- 201 Created; code stored (matching is case-insensitive on use).

How to test manually:

```
curl -i -X POST http://localhost:8080/admin/discount-codes \
-H "Authorization: Bearer $ADMIN_TOKEN" -H "Content-Type: application/json" \
-d '{"code":"SAVE20","type":"PERCENT","value":20,"maxDiscount":100,"minAmount":300,"active":true}'
```

P0 TC-5.4 Discount computation rules (unit)	
Type	Unit
Preconditions	None (DiscountServiceTest).

Steps:

1. Exercise blank code, percent, flat, cap, min-amount, inactive, unknown.

Expected result:

- Blank/null code -> 0 discount.
- PERCENT applies value%, capped by maxDiscount.
- FLAT applies fixed value.
- Discount never exceeds order amount.
- Below minAmount / inactive / unknown -> INVALID_DISCOUNT (or 0 per rules).

How to test manually:

```
mvn -Dtest=DiscountServiceTest test
```

P0 TC-5.5 Refund policy tiers (unit)	
Type	Unit
Preconditions	None (RefundPolicyServiceTest).

Steps:

1. Compute refund for various hours-before-show on a Rs.500 booking.

Expected result:

- >=24h before -> 100% (Rs.500).
- >=4h and <24h -> 50% (Rs.250).
- <4h -> 0%.

How to test manually:

```
mvn -Dtest=RefundPolicyServiceTest test
```

10. Level 6 - Show Management & Public Browsing

P0 TC-6.1 Create a show and materialize seats

Type	Functional
Preconditions	ADMIN token; movie + screen (with seats) exist.

Steps:

1. POST /admin/shows with movieId, screenId, startTime, basePrice.

Expected result:

- 201 Created; a show_event row created.
- One show_seat row per physical seat, status=AVAILABLE, price computed by PricingService.

How to test manually:

```
curl -i -X POST http://localhost:8080/admin/shows \
  -H "Authorization: Bearer $ADMIN_TOKEN" -H "Content-Type: application/json" \
  -d '{"movieId":1,"screenId":1,"startTime":"2027-03-01T18:00:00","basePrice":200}'
```

P1 TC-6.2 Weekend pricing applied at show creation

Type	Functional
Preconditions	ADMIN token; startTime on a Saturday/Sunday (in pricing.zone).

Steps:

1. Create a show whose startTime falls on a weekend.
2. GET /shows/{id}/seats.

Expected result:

- show_seat.price includes the weekend multiplier (default 1.25x).

How to test manually:

```
curl -s http://localhost:8080/shows/{showId}/seats | more
```

P1 TC-6.3 Public browse all shows

Type	Functional
Preconditions	At least one show exists.

Steps:

1. GET /shows (no auth).

Expected result:

- 200 OK; list of ShowDto.Response.

How to test manually:

```
curl -s http://localhost:8080/shows
```

P2 TC-6.4 Filter shows by city / movie / date range

Type	Functional
Preconditions	Shows exist across cities and dates.

Steps:

1. GET /shows?cityId=..&movieId=..&from=..&to=.. (ISO date-time).

Expected result:

- 200 OK; only shows matching all provided filters are returned.

How to test manually:

```
curl -s "http://localhost:8080/shows?cityId=1&movieId=1&from=2027-01-01T00:00:00&to=2027-12-31T23:59:59"
```

P1	TC-6.5 Get a single show and its seat map
Type	Functional
Preconditions	A valid showId.

Steps:

1. GET /shows/{id} then GET /shows/{id}/seats.

Expected result:

- 200 OK; seat view lists each show-seat id, label, class, price and status.

How to test manually:

```
curl -s http://localhost:8080/shows/1
```

```
curl -s http://localhost:8080/shows/1/seats
```

P2	TC-6.6 Get non-existent show
Type	Negative
Preconditions	None.

Steps:

1. GET /shows/999999.

Expected result:

- 404 Not Found; code=NOT_FOUND.

How to test manually:

```
curl -i http://localhost:8080/shows/999999
```

11. Level 7 - Booking Lifecycle (Happy Paths)

The core value: hold seats, confirm payment, and cancel with refund. All CUSTOMER-only and @Transactional.

P0 TC-7.1 Hold a single available seat

Type	Functional
Preconditions	CUSTOMER token; a showId with an AVAILABLE show-seat id.

Steps:

1. POST /shows/{showId}/holds with one showSeatId.

Expected result:

- 201 Created; booking status=PENDING; holdExpiresAt = now + ttl (120s).
- show_seat -> HELD with heldByUserId set; booking_seat active=true.
- totalAmount / finalAmount = that seat's price.

How to test manually:

```
curl -i -X POST http://localhost:8080/shows/1/holds \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"showSeatIds":[10]}'
```

P1 TC-7.2 Hold multiple seats (amount is the sum)

Type	Functional
Preconditions	CUSTOMER token; several AVAILABLE seats in one show.

Steps:

1. POST holds with a list of showSeatIds.

Expected result:

- 201; all listed seats -> HELD in one atomic transaction.
- finalAmount equals the sum of the seat prices.
- Seat ids are locked in ascending order (deterministic).

How to test manually:

```
curl -i -X POST http://localhost:8080/shows/1/holds \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"showSeatIds":[11,12,13]}'
```

P0 TC-7.3 Confirm a held booking (payment)

Type	Functional
Preconditions	A PENDING booking owned by \$TOKEN, hold not expired.

Steps:

1. POST /bookings/{bookingId}/confirm (optional paymentMethod).

Expected result:

- 200 OK; booking status=CONFIRMED; confirmedAt set.
- Seats -> BOOKED; hold fields cleared.
- Payment row status=SUCCESS, method from request or 'MOCK'.
- A CONFIRMED notification fires AFTER commit (async, in logs).

How to test manually:

```
curl -i -X POST http://localhost:8080/bookings/1/confirm \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"paymentMethod":"CARD"}'
```

P1	TC-7.3a Preview applicable discount coupons before confirming
Type	Functional
Preconditions	A PENDING booking owned by \$TOKEN; one or more active discount codes exist (e.g. SAVE20).

Steps:

1. GET /bookings/{bookingId}/discounts.
2. Inspect the returned coupon options (code, discountAmount, finalAmount).

Expected result:

- 200 OK; totalAmount echoes the booking's seat-price total.
- options lists ONLY coupons the customer can actually apply now (active, within validity window, order >= minAmount).
- Each option shows discountAmount and the resulting finalAmount; sorted best-saving first.
- Coupons below their minimum (e.g. order < minAmount) are omitted.
- Read-only: booking stays PENDING; the customer then picks one and confirms (TC-7.4).
- 409 CONFLICT if the booking is no longer PENDING (already confirmed/cancelled/expired).

How to test manually:

```
curl -i http://localhost:8080/bookings/1/discounts \
  -H "Authorization: Bearer $TOKEN"
# Response: { bookingId, totalAmount, options: [ {code, type, value, discountAmount, finalAmount} ] }
```

P1	TC-7.4 Confirm with a valid discount code
Type	Functional
Preconditions	PENDING booking with finalAmount >= discount minAmount; SAVE20 active (see preview TC-7.3a).

Steps:

1. POST confirm with discountCode=SAVE20 (the code chosen from the preview).

Expected result:

- 200; discount applied (20% capped at maxDiscount=100).
- Payment amount = finalAmount - discount.
- The applied discount matches what TC-7.3a previewed for that code.

How to test manually:

```
curl -i -X POST http://localhost:8080/bookings/1/confirm \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"discountCode":"SAVE20","paymentMethod":"CARD"}'
```

P0	TC-7.5 Cancel confirmed booking - full refund
Type	Functional
Preconditions	A CONFIRMED booking >= 24h before show start.

Steps:

1. POST /bookings/{bookingId}/cancel.

Expected result:

- 200; booking status=REFUNDED (refund > 0).
- Seats -> AVAILABLE; booking_seat active=false (rows not deleted).
- Payment status=REFUNDED; seat becomes re-holdable.

How to test manually:

```
curl -i -X POST http://localhost:8080/bookings/1/cancel \
  -H "Authorization: Bearer $TOKEN"
```

P1	TC-7.6 Cancel - partial refund window
Type	Functional
Preconditions	CONFIRMED booking between 4h and 24h before show.

Steps:

1. POST cancel.

Expected result:

- 200; refund = 50% (partialRefundPercent).
- Payment status=PARTIALLY_REFUNDED; booking status=REFUNDED.

How to test manually:

```
curl -i -X POST http://localhost:8080/bookings/{id}/cancel \
-H "Authorization: Bearer $TOKEN"
```

P1	TC-7.7 Cancel - no-refund window
Type	Functional
Preconditions	CONFIRMED booking < 4h before show.

Steps:

1. POST cancel.

Expected result:

- 200; refund = 0; booking status=CANCELLED (not REFUNDED).
- Payment stays SUCCESS; seats still released to AVAILABLE.

How to test manually:

```
curl -i -X POST http://localhost:8080/bookings/{id}/cancel \
-H "Authorization: Bearer $TOKEN"
```

P2	TC-7.8 List my bookings / get one booking
Type	Functional
Preconditions	CUSTOMER token with at least one booking.

Steps:

1. GET /bookings/me then GET /bookings/{id}.

Expected result:

- 200; only the caller's bookings are returned.

How to test manually:

```
curl -s http://localhost:8080/bookings/me -H "Authorization: Bearer $TOKEN"
```

```
curl -s http://localhost:8080/bookings/1 -H "Authorization: Bearer $TOKEN"
```

12. Level 8 - Booking Negative & Edge Cases

P0 TC-8.1 Hold a seat that is already HELD/BOOKED

Type	Negative
Preconditions	A seat currently HELD by another booking.

Steps:

1. POST holds for that show-seat id.

Expected result:

- 409 Conflict; code=SEAT_UNAVAILABLE; no state change.

How to test manually:

```
curl -i -X POST http://localhost:8080/shows/1/holds \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"showSeatIds":[10]}' # 10 already held
```

P1 TC-8.2 Hold seat that belongs to a different show

Type	Negative
Preconditions	A valid show-seat id that is NOT part of {showId}.

Steps:

1. POST /shows/{showId}/holds with a foreign show-seat id.

Expected result:

- 400 Bad Request; code=VALIDATION_ERROR (seat-show mismatch).

How to test manually:

```
curl -i -X POST http://localhost:8080/shows/1/holds \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"showSeatIds":[99999]}'
```

P0 TC-8.3 Confirm after hold expired

Type	Negative
Preconditions	PENDING booking whose holdExpiresAt is in the past.

Steps:

1. POST confirm on the expired booking.

Expected result:

- 409 Conflict; code=HOLD_EXPIRED.
- Booking set to EXPIRED and seats released as a side-effect.

How to test manually:

Lower booking.hold.ttl-seconds (e.g. 2s), hold a seat, wait >2s, then:

```
curl -i -X POST http://localhost:8080/bookings/{id}/confirm -H "Authorization: Bearer $TOKEN"
```

P1 TC-8.4 Confirm a non-PENDING booking

Type	Negative
Preconditions	An already CONFIRMED (or CANCELLED) booking.

Steps:

1. POST confirm again.

Expected result:

- 409 Conflict; code=CONFLICT (illegal state transition).

How to test manually:

```
curl -i -X POST http://localhost:8080/bookings/1/confirm \
-H "Authorization: Bearer $TOKEN"
```

P1 TC-8.5 Cancel a non-CONFIRMED booking

Type	Negative
Preconditions	A PENDING or EXPIRED booking.

Steps:

1. POST cancel.

Expected result:

- 409 Conflict; code=REFUND_NOT_ALLOWED.

How to test manually:

```
curl -i -X POST http://localhost:8080/bookings/{pendingId}/cancel \
-H "Authorization: Bearer $TOKEN"
```

P0 TC-8.6 Access another user's booking

Type	Security / Negative
Preconditions	Booking owned by user A; call with user B's token.

Steps:

1. GET /bookings/{A_bookingId} as user B (also try confirm/cancel).

Expected result:

- 403 Forbidden; code=FORBIDDEN (ownership enforced in service).

How to test manually:

```
curl -i http://localhost:8080/bookings/1 \
-H "Authorization: Bearer $OTHER_TOKEN"
```

P2 TC-8.7 Confirm with invalid/expired discount code

Type	Negative
Preconditions	PENDING booking.

Steps:

1. POST confirm with a non-existent or below-min discount code.

Expected result:

- 400 Bad Request; code=INVALID_DISCOUNT; booking stays PENDING.

How to test manually:

```
curl -i -X POST http://localhost:8080/bookings/1/confirm \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"discountCode":"NOPE999}"'
```

P2 TC-8.8 Hold on non-existent show

Type	Negative
Preconditions	CUSTOMER token.

Steps:

1. POST /shows/999999/holds.

Expected result:

- 404 Not Found (or VALIDATION_ERROR) - never a 500.

How to test manually:

```
curl -i -X POST http://localhost:8080/shows/999999/holds \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"showSeatIds":[1]}'
```

13. Level 9 - Hold Expiry

Holds must not leak seats. Expiry happens two ways: the background scheduler and the confirm-time check.

P0 TC-9.1 Scheduler expires an abandoned hold

Type	Functional / Timing
Preconditions	Short TTL for the test (e.g. ttl-seconds=1, sweep=300ms as in HoldExpiryTest).

Steps:

1. Hold a seat and do nothing.
2. Wait past TTL + one sweep interval.
3. Re-inspect booking and seat.

Expected result:

- Booking status -> EXPIRED.
- show_seat -> AVAILABLE; booking_seat active=false.
- Audit records HOLD_EXPIRED_SWEEP.

How to test manually:

Automated: mvn -Dtest=HoldExpiryTest test

Manual: set ttl-seconds low, hold via curl, wait, then GET /shows/{id}/seats and confirm the seat is AVAILABLE again.

P1 TC-9.2 Expired seat is immediately re-holdable

Type	Functional
Preconditions	A seat that just expired (TC-9.1).

Steps:

1. POST a new hold on the same show-seat id from any customer.

Expected result:

- 201 Created; the new hold succeeds (partial unique index allows re-book).

How to test manually:

```
curl -i -X POST http://localhost:8080/shows/1/holds \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"showSeatIds":[10]}'
```

14. Level 10 - Concurrency (Advanced, Highest Value)

These prove the core invariant: NO SEAT IS EVER SOLD TWICE. They combine the pessimistic lock (SELECT ... FOR UPDATE in id order), the @Version optimistic checks, and the partial unique DB index as a final backstop.

P0	TC-10.1 N buyers race for one seat - exactly one wins
Type	Concurrency (crown jewel)
Preconditions	One AVAILABLE seat; N=20 threads (SeatBookingConcurrencyTest).

Steps:

1. Spawn 20 threads that each try hold+confirm on the same show-seat.
2. Await all; count outcomes; inspect final DB state.

Expected result:

- Exactly 1 thread ends CONFIRMED; the other 19 fail with SEAT_UNAVAILABLE.
- Final show_seat status=BOOKED.
- Exactly 1 CONFIRMED booking and 1 active booking_seat for that seat.
- No duplicate/double-sell; no deadlock; no 500s.

How to test manually:

Automated (authoritative): `mvn -Dtest=SeatBookingConcurrencyTest test`

Manual load approximation: fire many parallel holds and count 201 vs 409:

```
for i in $(seq 1 20); do curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8080/shows/1/holds -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -d '{"showSeatIds":[10]}' & done; wait
```

P1	TC-10.2 Concurrent holds on different seats all succeed
Type	Concurrency
Preconditions	Many AVAILABLE seats; each thread targets a distinct seat.

Steps:

1. Spawn threads each holding a different show-seat id.

Expected result:

- All holds succeed (no false contention - locks are per-row, in id order).
- No deadlock despite parallel FOR UPDATE.

How to test manually:

Fire parallel curls each with a different showSeatId; expect all 201.

P0	TC-10.3 DB unique index backstop prevents double active seat
Type	Concurrency / DB
Preconditions	Ability to force two active booking_seat rows for one show_seat.

Steps:

1. Attempt (via a race or direct insert) to create a second active booking_seat for the same show_seat_id.

Expected result:

- DataIntegrityViolation on uq_booking_seat_active -> mapped to 409 SEAT_UNAVAILABLE.
- Only one active row survives; invariant preserved even if app logic were bypassed.

How to test manually:

Direct DB proof:

```
psql moviebooking -c "insert into booking_seat(booking_id,show_seat_id,active) values (1,10,true),(2,10,true);"
# second row must be rejected by the partial unique index
```

P1	TC-10.4 Optimistic lock (@Version) rejects stale write	
Type	Concurrency	
Preconditions	Two transactions read the same ShowSeat/Booking version.	

Steps:

1. Both attempt to update; the second commit uses a stale version.

Expected result:

- OptimisticLockingFailureException -> 409 SEAT_UNAVAILABLE; no lost update.

How to test manually:

Covered indirectly by TC-10.1; verified via GlobalExceptionHandler mapping.

15. Level 11 - End-to-End Journeys & Non-Functional

P0 TC-11.1 Full admin-to-customer journey

Type	End-to-end
Preconditions	Fresh app; admin and customer tokens.

Steps:

1. ADMIN: create city -> theater -> screen -> seats -> movie -> pricing tier -> show + discount code.
2. CUSTOMER: browse shows -> view seat map -> hold seats.
3. CUSTOMER: preview applicable coupons (GET /bookings/{id}/discounts) and pick the best one.
4. CUSTOMER: confirm with the chosen discount code -> verify payment.
5. CUSTOMER: cancel -> verify refund and seat release.
6. CUSTOMER: re-hold the released seat successfully.

Expected result:

- Every step returns the documented status.
- Money math is consistent end to end (price -> discount -> payment -> refund).
- Seat status transitions AVAILABLE -> HELD -> BOOKED -> AVAILABLE correctly.

How to test manually:

Chain the curl snippets from Levels 4-8 in order; or drive it in Swagger UI. Also see BookingLifecycleTest for the automated version.

P2 TC-11.2 Notification fires only after commit

Type	Non-functional / Async
Preconditions	App running; confirm a booking.

Steps:

1. Confirm a booking and watch application logs.

Expected result:

- Notification log line appears AFTER the transaction commits (AFTER_COMMIT).
- It runs on an async thread (not the request thread); a rolled-back tx sends nothing.

How to test manually:

Confirm via curl (TC-7.3) and tail the app logs for the async notification entry.

P1 TC-11.3 Error responses have a consistent shape

Type	Non-functional
Preconditions	None.

Steps:

1. Trigger several errors (401, 403, 404, 409, 400).

Expected result:

- Every error body has timestamp, status, code, message, path.
- Validation errors additionally include fieldErrors.
- No stack traces or internal details leak to the client.

How to test manually:

Compare bodies from TC-1.3, TC-2.1, TC-6.6, TC-8.1 - all share the ApiError schema.

P0	TC-11.4 Full regression suite is green
Type	Non-functional / CI
Preconditions	Zonky embedded PostgreSQL (no Docker needed).

Steps:

1. Run the entire test suite.

Expected result:

- mvn test passes: unit (pricing/discount/refund) + integration (lifecycle, expiry, RBAC) + concurrency race.

How to test manually:

```
mvn test
```

16. Traceability Matrix

Maps automated tests in the repository to the manual test cases above.

Automated test	Covers TC(s)
SeatBookingConcurrencyTest	TC-10.1, TC-10.4
BookingLifecycleTest	TC-7.1, 7.3, 7.5, TC-11.1
BookingDiscountPreviewTest	TC-7.3a
HoldExpiryTest	TC-9.1, TC-9.2
RbacAndValidationTest	TC-1.3, TC-2.1, TC-2.2, TC-2.7, TC-4.1
PricingServiceTest	TC-5.2
DiscountServiceTest	TC-5.4, TC-7.3a
RefundPolicyServiceTest	TC-5.5
(manual/exploratory)	TC-0.x, TC-6.x, TC-8.x, TC-10.2/10.3, TC-11.2/11.3

16.1 Coverage summary by level

Level	Theme	# TCs
0	Smoke & environment	4
1	Authentication	8
2	Authorisation / RBAC	7
3	Input validation	3 (+matrix)
4	Admin catalog	5
5	Pricing / discount / refund	5
6	Shows & browsing	6
7	Booking lifecycle (happy)	9
8	Booking negative/edge	8
9	Hold expiry	2
10	Concurrency (advanced)	4
11	End-to-end & non-functional	4

End of document. To keep this catalogue current, regenerate the PDF after changing endpoints or business rules: `python docs/generate_test_cases_pdf.py`